

Énoncé — Projet final : Pipeline Big Data temps réel

Durée : 24h · Équipe · Environnement local (laptop 16 Go)

Objectif

Concevoir et implémenter un pipeline **Kappa** : Kafka → Spark Structured Streaming → Delta Lake (Bronze/Silver/Gold). Un seul chemin de traitement, pas de couche batch séparée.

Architecture attendue

Un générateur d'événements alimente Kafka. Spark Structured Streaming consomme le flux et écrit en continu vers une table Delta Bronze. Silver nettoie et déduplique. Gold restitue les données sous forme de **schéma en étoile** (table de faits + tables de dimension), directement interrogeable par un outil de BI. Au moins un usage démontré de `MERGE INTO` ou de time travel.

Cas d'usage imposé

Monitoring d'une flotte de capteurs industriels. Ce cas est imposé parce qu'il oblige un vrai flux continu — contrairement à un clickstream ou des transactions, qui peuvent être simulés par lots sans que ça se voie, un capteur qui n'émet pas en continu n'a pas de sens.

Le générateur simule un site industriel avec plusieurs dizaines de capteurs (température, vibration, pression) répartis sur quelques machines, chacun émettant une mesure toutes les 1 à 3 secondes.

Schéma de l'événement Kafka :

```
{
  "event_id": "evt-9f3a1c2d",
  "capteur_id": "cpt-042",
  "machine_id": "m-07",
```

```

"site_id": "site-lyon",
"type_mesure": "temperature",
"valeur": 78.4,
"unite": "celsius",
"qualite_signal": 0.97,
"batterie_pourcentage": 63,
"timestamp": "2026-07-08T10:15:32.104Z"
}

```

Référentiel statique (données de référence) :

En complément du flux Kafka, vous devez produire vous-même un référentiel statique décrivant les capteurs, machines et sites de votre simulation — cohérent avec les identifiants (`capteur_id` , `machine_id` , `site_id`) utilisés par votre générateur. À vous de l'exploiter dans votre modélisation Gold. Exemples de lignes attendues :

`capteurs.csv` — `capteur_id`, `type_mesure`, `plage_nominale_min`, `plage_nominale_max`, `fabricant`, `date_installation`, `precision_capteur`

```

cpt-042, temperature, 10, 85, Siemens, 2023-03-14, 0.5
cpt-017, vibration, 0, 12, Bosch, 2022-11-02, 0.1

```

`machines.csv` — `machine_id`, `type_machine`, `ligne_production`, `criticité`, `date_mise_service`, `capacite_nominale`, `responsable_technique`

```

m-07, presse hydraulique, ligne-A, haute, 2021-06-01, 500, J. Dupont
m-12, convoyeur, ligne-B, moyenne, 2020-09-15, 1200, S. Martin

```

`sites.csv` — `site_id`, `nom`, `région`, `capacite_site`, `fuseau_horaire`, `responsable_site`

```

site-lyon, Lyon Usine 1, Auvergne-Rhône-Alpes, 3000, Europe/Paris, M. Bernard
site-nantes, Nantes Usine 2, Pays de la Loire, 1800, Europe/Paris, L. Petit

```

Le référentiel ne change pas (ou très peu) pendant l'exécution ; le générateur reste la seule source de faits en flux continu.

Attendus fonctionnels minimaux :

- **Bronze** : capture brute du flux, sans perte
- **Silver** : détection d'anomalie par seuil (ex. température > 85°C, vibration au-delà d'un seuil défini par machine) — les événements hors seuil sont marqués, pas supprimés
- **Gold** : modélisé en **schéma en étoile** — à vous de définir la table de faits et les tables de dimension (granularité, clés, attributs) à partir du flux et du référentiel fourni. Doit couvrir au minimum :
 - Une agrégation temporelle (moyenne/max glissants par machine sur une fenêtre de quelques minutes), alimentée en continu — justifie le traitement en streaming
 - Une table d'**état courant par capteur** (dernière valeur connue, dernier statut), mise à jour par `MERGE INTO` à chaque nouvel événement — c'est l'usage le plus naturel et le plus utilisé en production de ce mécanisme, à ne pas remplacer par un simple `append`

Les deux mécanismes Delta (`append` en streaming, `MERGE INTO`) doivent rester visibles et justifiés séparément dans le README.

Le générateur exact (script Python publiant vers Kafka) sera fourni avec les paramètres (nombre de capteurs, machines, fréquence, taux d'anomalies injectées).

Restitution BI (Metabase)

Le Gold doit être connecté à **Metabase** (conteneur Docker additionnel, léger) avec au moins un dashboard exposant des KPIs pertinents pour un responsable de site industriel.

Livrables

Dépôt Git avec l'intégralité du code (docker-compose, jobs Spark, générateur) et un **README qui prouve et explique chaque étape**. Structure attendue :

1. **Architecture** — schéma ou description de votre implémentation réelle (pas celle du sujet), avec les choix de partitionnement et de format des tables
2. **Lancement** — commandes exactes et reproductibles, de zéro à pipeline qui tourne (`docker compose up` , soumission des jobs Spark, lancement du générateur)
3. **Preuves** — pour chaque étage (Kafka → Bronze → Silver → Gold), une capture ou une sortie de commande montrant que les données transitent réellement, plus une requête `DESCRIBE HISTORY` ou `VERSION AS OF` exécutée avec succès, plus une capture du dashboard Metabase avec les KPIs à jour
4. **Justifications** — pourquoi Kappa dans ce contexte, comment le checkpoint est géré, ce que vous feriez pour la compaction des petits fichiers si vous aviez plus de temps
5. **Incident** — un problème réellement rencontré (checkpoint cassé, OOM, désynchronisation de schéma...) et comment vous l'avez résolu

Pas de rapport séparé. Le README est le document de référence — s'il ne suffit pas à comprendre, reproduire et faire tourner le projet, il est incomplet.

Soutenance : 15 minutes de présentation + 10 minutes de questions. Le pipeline doit être exécutable en direct si demandé.

Notation

- Pipeline fonctionnel bout en bout
- Fonctionnement correct de Kafka et Spark
- Usage correct de Delta
- Pertinence de la modélisation en étoile et des KPIs Metabase
- Qualité de la preuve et de la justification dans le README
- Clarté de la soutenance et solidité des réponses aux questions.